Geavanceerde Computerarchitectuur Project
Raycasting

Christoph Sanders & Wout Corijn

2025-04-28

1 Inleiding

In dit project vergelijken we de prestaties van CPU's en GPU's aan de hand van een raycaster die voor beide hardware componenten implementeren. Raycasting is een computationeel intensieve techniek waardoor het een ideaal testgeval vormt voor deze vergelijking. We starten met een korte uitleg van de benodigde wiskunde achter raycasting. Vervolgens bespreken we enkele implementatiedetails. Tot slot analyseren we de resultaten van beide methoden.

2 Background

2.1 Raycaster

2D raycasting is een techniek voor het simuleren van bronnen die signalen uitzenden, vergelijkbaar met hoe de zon lichtstralen uitzendt. Voor elke straal (ray) wordt berekend waar deze het eerste object in de omgeving raakt. Deze vorm van raycasting vormt de basis voor diverse toepassingen en kan uitgebreid worden met effecten zoals reflectie, fading en refractie om complexer gedrag van signalen of licht na te bootsen.

Naast visualisaties wordt raycasting ook toegepast in onder andere het genereren van eenvoudige 3D-beelden, zoals in het klassieke computerspel *Wolfenstein 3D* (1992) [4], en in het modelleren van fysische fenomenen zoals de verspreiding van elektromagnetische golven, bijvoorbeeld wifi-signalen.

Voor de raycaster die in deze thesis ontwikkeld is, hebben we ons beperkt tot reflectie op cirkels en het modelleren van signaalverzwakking (fading). Voor het berekenen van de afstand tot cirkels is gebruikgemaakt van de wiskundige methode zoals uitgelegd in een video van Chris Marriott [1].

2.2 Reflectie

De reflectie van een lichtstraal op het oppervlak van een bol wordt bepaald door de reflectiewet: de invalshoek is gelijk aan de uitvalshoek, gemeten ten opzichte van de normaal op het raakpunt. Bij een bol is deze normaal de vector van het middelpunt van de bol naar het raakpunt. Dit betekent dat de invalshoek en de uitvalshoek gelijk zijn ten opzichte van deze normaal [2].

De gereflecteerde vector wordt berekend aan de hand van Formule 1 [3].

$$r = d - 2(d \cdot n)n \tag{1}$$

Hierbij is:

- **d** de genormaliseerde richting van de invallende straal
- **n** de genormaliseerde normaal op het raakpunt (van het middelpunt naar het raakpunt)
- **r** de richting van de gereflecteerde straal

2.3 Fading

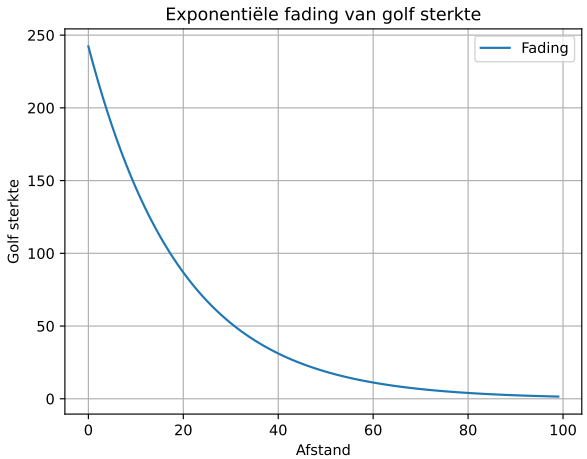
Wanneer lichtgolven zich verwijderen van een bron, neemt hun intensiteit exponentieel af. Dit fenomeen, bekend als 'fading', speelt een belangrijke rol bij het modelleren van realistische licht- en signaalverspreiding. In onze simulatie hebben we ervoor gekozen om dit gedrag na te bootsen via een eenvoudige exponentiële functie, zodat de intensiteit van de lichtstralen (rays) afneemt naarmate ze zich verder verspreiden of reflecteren (zie codevoorbeeld in Listing 1).

```
double fadeLength = 16;
double fadeFactor = 0.998;
double fadeByte = fadeLength * (ray.pixel[0] >> 24);

for (int j = 0; j < ray.length[rayIndex]; j++) {
        fadeByte = fadeByte*fadeFactor;
        ...
}
```

Listing 1: Fading code

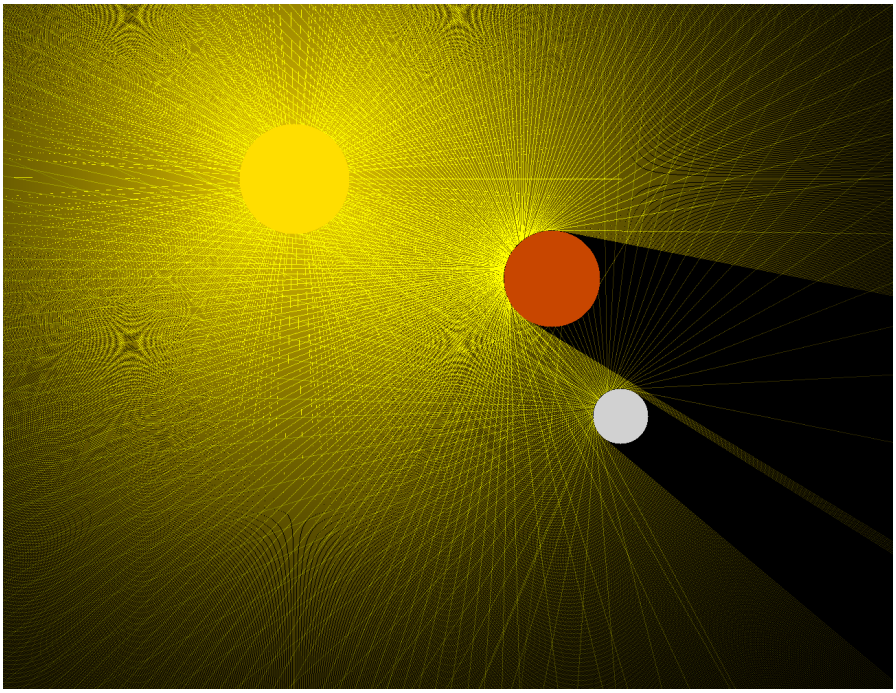
De gebruikte formule resulteert in een typische exponentiële afname, wat ook bevestigd wordt door de metingen weergegeven in Figuur 1. Hierin is te zien hoe de intensiteit van een straal snel afneemt bij toenemende afstand vanaf de bron.



Figuur 1: Fading verloop

3 Context

In dit project wordt raycasting gebruikt om de zonnestraling op twee planeten te simuleren. Daarbij wordt ook reflectie in rekening gebracht, waardoor het mogelijk wordt om de weerkaatsing van het licht op de planeten te modelleren. De simulatie wordt zowel op de CPU als op de GPU opgebouwd om een vergelijking te kunnen maken tussen beide implementaties.



Figuur 2: Raycaster screenshot

4 Implementatie

De rendering wordt uitgevoerd met behulp van SDL3. De keuze voor SDL3 is gemaakt vanwege de mogelijkheid met het werken met matrices. SDL3 maakt het mogelijk om een vooraf geïnitialiseerd display's aan te sturen door middel van matrixoperaties, waarbij elke cel in de matrix overeenkomt met een pixel op het scherm. Elke cel bevat gegevens die de kleurwaarde van die pixel voorstellen, doorgaans gecodeerd in een 32-bits formaat waarin de intensiteit van rood, groen, blauw en transparantie wordt opgeslagen.

Voor elk frame worden eerst alle pixels zwart gemaakt. In de GPU-versie gebeurt dit parallel: de matrix die de pixels representeert, wordt in het globale geheugen van de GPU geladen. Dit deel van het geheugen is toegankelijk voor alle aangemaakte threads, zelfs als ze zich niet in hetzelfde block bevinden. Er worden evenveel threads als pixels aangemaakt, gegroepeerd in blocks van 1024 threads. Elke thread schrijft de waarde 0 naar de overeenkomstige cel in de matrix; hiervoor is geen synchronisatie nodig. Bij de CPU-versie daarentegen wordt deze operatie sequentieel uitgevoerd. Het hoofdproces doorloopt iteratief alle pixels (cellen in de matrix) en schrijft de waarde 0 naar elke individuele pixel. Deze aanpak is eenvoudiger, maar minder efficiënt bij grote resoluties.

Daarna worden de cirkels op het scherm getekend. In de GPU-implementatie wordt opnieuw de matrix in het globale geheugen geladen, terwijl de gegevens van de cirkels in het gedeelde (shared) geheugen worden geplaatst. Het inladen van de X aantal cirkels in het gedeelde geheugen is de verantwoordelijkheid van de eerste X threads van elk block. Op deze manier kan het lezen van de cirkeldata efficiënter verlopen. Elke thread controleert vervolgens of de bijbehorende pixel zich binnen de grenzen van een cirkel bevindt. Als dat het geval is, wordt de kleur van de cirkel naar de pixel gekopieerd. Dankzij de parallelle verwerking op de GPU kunnen al deze controles en kleuroperaties gelijktijdig worden uitgevoerd. In de CPU-versie wordt dit proces sequentieel uitgevoerd. Het hoofdproces doorloopt elke pixel één voor één en controleert telkens of de pixel zich binnen een cirkel bevindt. Indien dit zo is, krijgt de pixel de juiste kleur toegewezen.

Als laatste stap worden de stralen (rays) getekend die vertrekken vanuit de lichtbron (de zon). Voor elke straal wordt eerst het gepropageerde pad berekend. Dit houdt in dat de lengte van de straal en de reflectiehoeken die de straal ondervindt, worden berekend. Na elke reflectie wordt de nieuwe lengte opnieuw berekend en de reflectie. Om oneindige reflecties te voorkomen, wordt een maximaal aantal reflecties ingesteld. Nadat dit traject volledig is berekend, worden de bijbehorende stralen visueel weergegeven. Dit principe wordt als volgt geïmplementeerd:

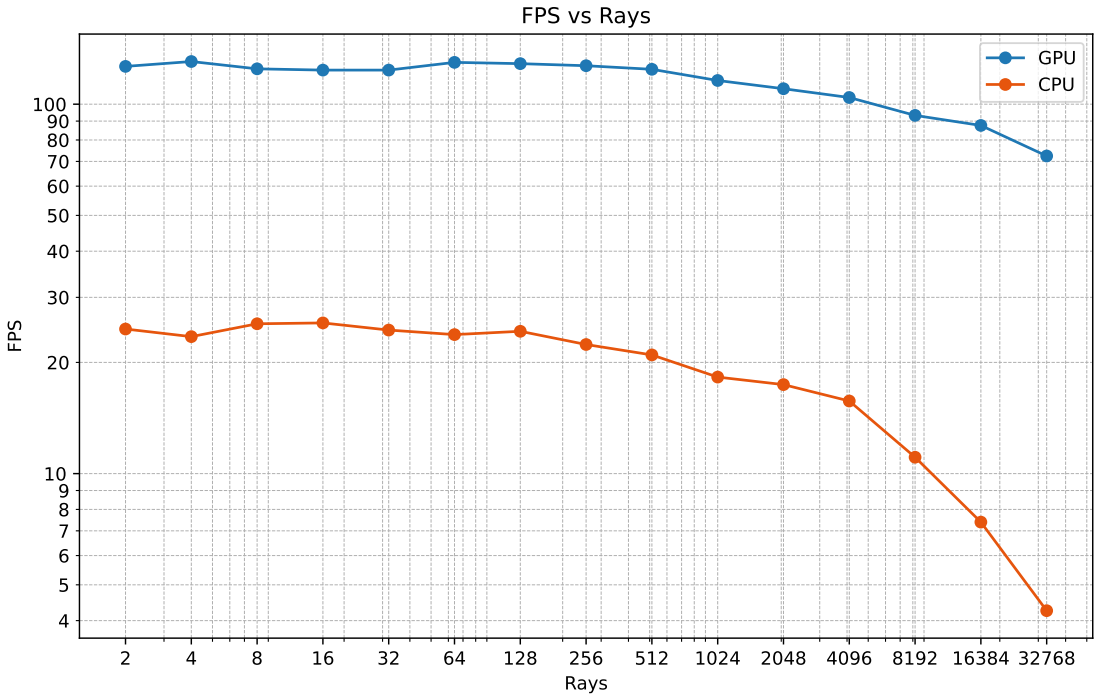
- **GPU**: De pixels (matrix) worden opnieuw in het globale geheugen geladen, de cirkels worden in het gedeelde geheugen geplaatst (ingeladen door de eerste threads van elk block), en de gegevens van de stralen (rays) worden eveneens in het globale geheugen geladen. Elke thread is verantwoordelijk voor de berekeningen van één straal. Daarnaast is elke thread verantwoordelijk voor het tekenen van zijn eigen straal. Dit gebeurt met behulp van atomische operaties om synchronisatiefouten te voorkomen. Deze aanpak is noodzakelijk omdat we constructieve interferentie tussen stralen simuleren en anders read-after-writeconflicten zouden tegenkomen.
- **CPU**: Bij de CPU-implementatie wordt elke straal (ray) sequentieel berekend, inclusief de lengte en de reflecties. Nadat alle stralen zijn berekend, worden ze één voor één getekend in de matrix.

5 Resultaten

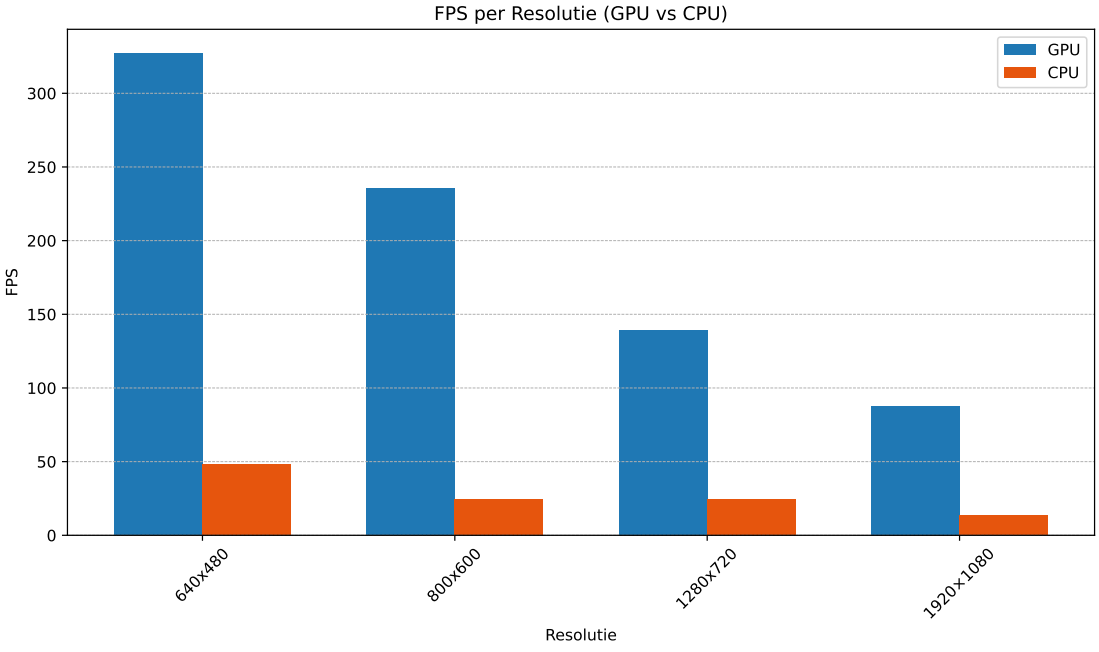
Bij de implementatie van de raycaster op zowel de GPU als de CPU is een vergelijking gemaakt tussen beide uitvoeringen. Hierbij is gevarieerd in het aantal rays en de displaygrootte. De impact van deze variaties is gemeten aan de hand van de gemiddelde FPS-waarden (zie Figuur 4). Deze waarden zijn gebaseerd op 5000 cycli, waarbij elke cyclus bestaat uit het berekenen en tekenen van alle objecten en rays op het scherm.

Uit de grafiek blijkt dat de GPU aanzienlijk beter presteert dan de CPU. Dit is logisch, aangezien bij een laag aantal rays de CPU al merkbaar trager is dan de GPU, omdat zowel het renderen van alle objecten als het tekenen op het scherm volledig door de CPU wordt uitgevoerd.

Naarmate het aantal rays toeneemt, daalt de FPS bij beide implementaties. De daling is bij de GPU sterker in absolute termen, maar de prestaties blijven aanzienlijk beter dan bij de CPU. Bij 32 768 rays zakt de CPU tot 4 FPS, terwijl de GPU op datzelfde punt nog 72 FPS haalt.



Figuur 3: FPS vs Rays (1280 × 1024)



Figuur 4: FPS vs windowsize

6 Conclusie

Dit project toont aan dat GPU's aanzienlijk beter presteren dan CPU's bij computationeel zware taken zoals raycasting. Dankzij parallelle architectuur voeren GPU's een groot aantal berekeningen tegelijk uit, wat leidt tot een veel hogere framerate. Hoewel programmeren op een GPU complexer is, levert het een aanzienlijke efficiëntiewinst op. Raycasting illustreert daarmee uitstekend hoe GPU-verwerking grote voordelen biedt bij intensieve rekentaken.

Github repo: `https://github.com/woutcorijn/projectGeavanceerdeComputerarchitectuur`

Referenties

[1] Chris Marriott. Finding the intersection point of a line and a circle (in javascript) - how to make a web game, 2021. Geraadpleegd op 5 mei 2025.

[2] Tom Weideman. 4.3: Spherical reflectors, November 2022. Geraadpleegd op 5 mei 2025.

[3] Wikipedia contributors. Ray tracing (graphics) — Wikipedia, the free encyclopedia, 2025. Geraadpleegd op 5 mei 2025.

[4] Wikipedia contributors. Wolfenstein 3d — Wikipedia, the free encyclopedia, 2025. [Online; accessed 12-May-2025].